

Embedded System Lab – Term Project Report

Topic: Air Taiko Game(空氣太鼓達人)

Group Member: B12901095 傅品翔、B12901143 吳柏均、B12901141 吳宗翰

Github Repository:

STM: https://github.com/jun16q/es_term_project

PC: https://github.com/megumin0507/ES_final

1. Motivation

Inspired by a previous student's air drum project, we found the concept very interesting and decided to expand upon it by developing it into a fully interactive game.

We are also fascinated by motion-sensing controllers such as the Nintendo Switch Joy-Con, so our goal is to develop a similar handheld controller that uses a three-axis accelerometer, combined with real-time Bluetooth transmission, to enable motion-based interaction.

Below is a demonstration of the Taiko no Tatsujin game on the Nintendo Switch.



2. Result (Demo: <https://youtu.be/v320wsblsno>)



The game features three distinct classes of notes, each corresponding to a specific action. Each action produces a corresponding sound when it is performed:

- ✧ Red Notes: Represent a "Down Hit".
- ✧ Blue Notes: Represent a "Side Hit".

- ✧ Yellow Notes: Exclusive to the Single-Player Mode, these represent a "Drum Roll" (Ren-da). When this note appears, the player must perform rapid, continuous hits within a set duration to accumulate score.

Our game supports two mode: 1P and 2P mode

- 1P game introduction

Just like the classic taiko game, notes will scroll horizontally from the right side of the screen, and the player has to perform the corresponding motion at the right timing.

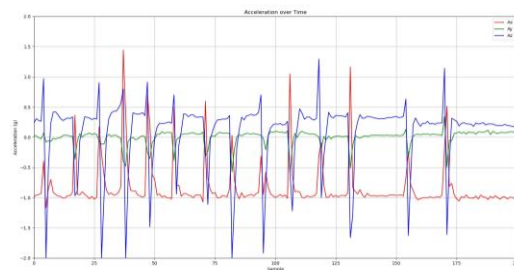
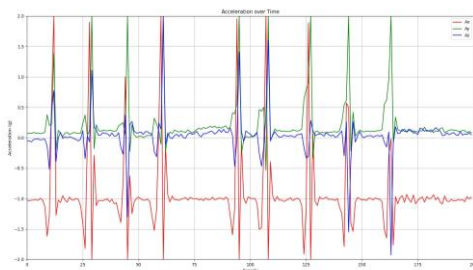
- 2P game introduction

A sequence of actions is first displayed on the screen, along with a specific rhythm. The player performs the actions according to the given rhythm, and after the actions are completed (or when the time runs out), the system evaluates the performance and then proceeds to the next round.

3. Implementation

Accelerometer Readings and Signal Processing

- The system utilizes the STM32 BSP drivers to periodically sample accelerometer data at a frequency of 50 Hz. Upon acquisition, the raw sensor data undergoes real-time analysis to classify user gestures into two distinct motion vectors: Vertical (Down) and Lateral (Side).
- The classification logic relies on a threshold-based algorithm. When the accelerometer magnitude on a specific axis exceeds a pre-defined threshold, the system registers a motion event. Also, a 150 ms debounce interval is applied immediately following a detected event to eliminate false positives.



- To facilitate precise gameplay mechanics, the interval between consecutive motion events is calculated using the HAL_GetTick() function. This timestamping mechanism ensures accurate latency measurement and rhythm detection.

BLE Peripheral Settings

- Objective: To transmit the detected Motion Class and the Time Interval since the last action.

- Framework: Adapted from the Lab 4 firmware structure.
- In gatt_db.c:
 - Implement a function to add a custom service
 - Set a 4-byte (uint32) characteristic
 - Implement a characteristic update function
- In app_bluenrg_ms.c:
 - In user_process(), call the update function upon detecting a new motion
 - The transmitted value is formed as: motionTime + (motionClass << 16). Using the same characteristic to transmit two data fields is more convenient and energy-efficient. This method works because motionTime will typically not exceed 65,535 (~65 seconds)
- A global (cross-file) variable is implemented in mydata.h to pass the motionClass value from main.c

RTOS Architecture (Built upon FreeRTOS)

- Sensor Task: Responsible for accelerometer data polling, signal classification, and interval calculation.
- Communication Task: Manages the Bluetooth Low Energy (BLE) stack to transmit processed motion flags and timing data to the host PC.

Latency Minimization Strategy

- A BLE connection interval is the time between two data transfer events between the central and the peripheral device. Initially, we tried to set the connection interval to a lower value, but the bleak module of python sometimes tries to reparameterize the value to 60 ms. We could not further reduce it. This large connection interval prevented real-time motion transmission and degraded the gaming experience.
- Then we applied a new technique on the STM32 side. Whenever the central device attempts to increase the interval, the STM32 immediately sends a connection parameter update request to force the interval back to the minimum of 7.5 ms. This approach successfully minimized the delay and ensured a responsive user experience.

```
case EVT_LE_CONN_UPDATE_COMPLETE:
{
    evt_le_connection_update_complete *cu = (void *)evt->data;

    if (cu->status == 0) {
        uint16_t new_interval = cu->interval;
        float interval_ms = new_interval;

        printf("Connection Parameters Updated!\n");
        printf("New Interval: %d \n", new_interval, interval_ms);
        aci_l2cap_connection_parameter_update_request(
            connection_handle,
            6, // Interval Min
            6, // Interval Max
            0, // Latency
            200 // Timeout (200 * 10ms = 2s)
        );
    }
}
break;
```

Combine Pygame with Bleak

- We met an error when trying to combine Pygame directly with Bleak:
“bleak.exc.BleakError: Thread is configured for Windows GUI but callbacks are not working.”
- This is because the Windows Bluetooth WinRT API must run in a Multi-Threaded Apartment (MTA) environment. In some setups, especially when mixing Python 3.13 + asyncio + GUI/pygame, the event loop and thread model may not meet WinRT’s requirements.
- We resolved the problem by putting Bleak in a separate thread, while still letting Pygame own the main thread.

```
def start_ble_thread(self):
    """
    Run the BLE asyncio loop in a background thread so pygame can own the main thread.
    """
    loop = asyncio.new_event_loop()

    def runner():
        asyncio.set_event_loop(loop)
        loop.run_until_complete(self.ble_main())
        loop.close()

    t = threading.Thread(target=runner, daemon=True)
    t.start()
```

4. References

https://www.youtube.com/watch?v=n3vrqPTn_wQ

<https://www.st.com/resource/en/datasheet/lsm6dsl.pdf>

https://www.st.com/resource/en/user_manual/um1873-getting-started-with-the-xcubeb1-bluetooth-low-energy-software-expansion-for-stm32cube-stmicroelectronics.pdf

<https://www.pygame.org/docs/>

<https://bleak.readthedocs.io/en/latest/>